

Loan Default Modeling — Analysis & Implementation

Concise walkthrough for data scientists: data overview, preprocessing, two classifiers (logistic regression & decision tree), model interpretation, borrower-level expected loss, and recommended next steps.



Dataset Snapshot

10000 rows × 8 columns. Core fields used:

- credit_lines_outstanding
- loan_amt_outstanding
- total_debt_outstanding
- income
- years_employed
- fico_score
- default (target)

customer_id removed before modeling. Quick sanity checks showed reasonable ranges and class imbalance toward non-defaults.

Loan	Cpain	Loan	Loan	Loan
cd999	#C4a46e	#c4a070	#249Z6	#83999
#14	#C4a46e	fca9070	#8a900	#B.900
#10	#f3a073	#3.5070	#89090	#G956
#00	#6fb4C7	#39578	#a297.8	#b79B
#50	#6fb4C3	#a4076	#22936	#0936
#1B	#a99b7cc	#a9aafbe	#a2956	# 0050
#60	#flab1C3	#83973	#23960	#b.365
#51	#afb44C7	#32973	#227899	#D676
#58	#6-99C8	#63078	#949899	#B0B5
#93	#ef41497	#89978	/29486	#B689B
#30	#AFa480	#90966	#9fb4C6	#FD796
#B99.99	#AFA7A0	#1.30	#AFA7A0	#AFA7A0

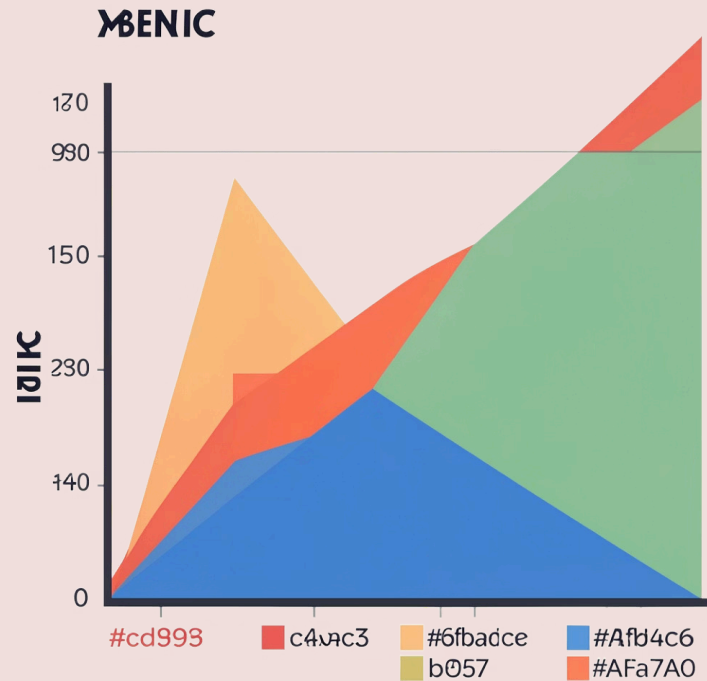


Preprocessing & Pipeline

Steps implemented:

- Drop `customer_id` and target from features
- `StandardScaler` fitted on training features
- Train/test split: 70% train / 30% test (`random_state=42`)
- Models: `LogisticRegression` and `DecisionTreeClassifier`

❏ Note: scaler was fitted on a `DataFrame` (with feature names).
Passing raw lists to scaler triggered warnings — use a `DataFrame` or maintain consistent feature ordering.



Logistic Regression — Key Results

Model trained on scaled features. Predicted probabilities produced sensible dynamic range, but class predictions were highly skewed toward non-default (precision/recall trade-offs need review).

Pros

Interpretable coefficients; fast inference; probability outputs for risk scoring.

Cons

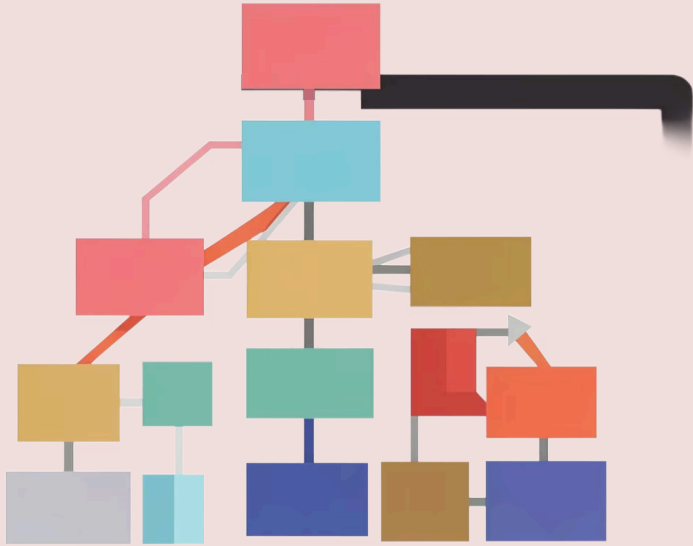
Linear decision boundary; sensitive to feature scaling and multicollinearity.



Confusion Matrix — Logistic Regression

Confusion matrix computed on test set. Observed very low true positive detection for defaults — suggests threshold tuning, class reweighting, or resampling (SMOTE/undersampling).

- Action: evaluate precision-recall curve and pick threshold by business trade-off
- Action: try `class_weight='balanced'` or sample rebalancing during training



Decision Tree — Interpretability & Leaves

DecisionTreeClassifier trained (random_state=42). Visualized top levels (max_depth=3) for inspection. Leaf counts indicate several pure default leaves and many majority-non-default leaves — tree captures non-linear splits relevant to default.

1

¹Advantage

Captures non-linear interactions and simple rules (easy to translate to business rules).

2

²Considerations

Prone to overfitting; prefers pruning or max_depth constraints; unstable with small data changes.

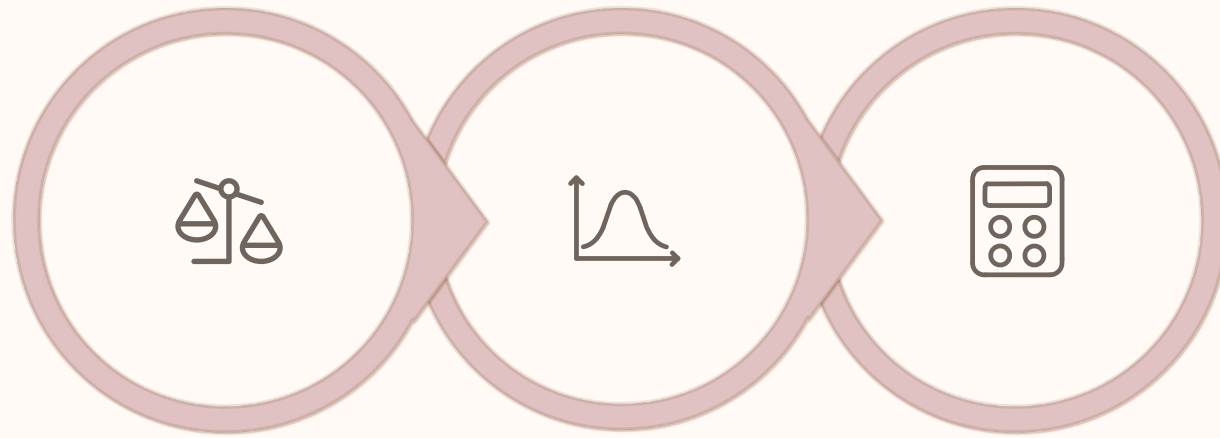


Feature Importance & Coefficients

Logistic regression coefficients (sorted):

- credit_lines_outstanding: +8.36 (strong positive)
- total_debt_outstanding: +3.90 (positive)
- loan_amt_outstanding: +0.17 (small positive)
- fico_score: -1.18 (protective)
- income: -2.47 (protective)
- years_employed: -2.73 (protective)

Interpretation: more credit lines and higher total debt raise default odds; higher income, tenure, and FICO reduce it. Check multicollinearity between loan_amt and total_debt.



**Scale
Features**

Predict PD

Compute EL

This function accepts a feature vector [credit_lines_outstanding, loan_amt_outstanding, total_debt_outstanding, income, years_employed, fico_score], scales it, predicts PD, and multiplies by Exposure at Default (loan_amt_outstanding) and LGD (1 - recovery_rate).



Example — Borrower Expected Loss

Borrower features: [5, 1958.93, 8228.75, 26648.44, 2, 572]

Results:

- Logistic model expected loss ≈ 1763.04 (units same as loan_amt)
- Decision tree expected loss ≈ 1763.04 (same function reused; note: PD from tree is discrete)

i Caveat: scaler warnings occurred because a raw list (no feature names) was passed to a scaler fitted on a DataFrame. Fix by converting the input to a DataFrame with matching column names before transforming.

Recommendations & Next Steps



1. Improve class handling

Apply `class_weight`, threshold optimization, or SMOTE; evaluate PR-AUC and cost-sensitive metrics aligned with expected loss.

2. Robust validation

Use cross-validation with stratification and time-based splits if appropriate; calibrate probabilistic outputs (Platt/ isotonic).

3. Model governance

Archive model artifacts, store scaler feature names, log drift metrics, and produce Explainable AI outputs (SHAP) for feature-level explanations.

4. Productionize

Wrap preprocessing in a pipeline (scaler + model), add input validation, and create an API that returns PD and expected loss per borrower.